

# **Отчёт по лабораторной работе №8**

**Целочисленная арифметика многократной точности**

Каймакджыоглу Мерич Дорук

# Содержание

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                    | <b>5</b>  |
| <b>2</b> | <b>Задание</b>                        | <b>6</b>  |
| <b>3</b> | <b>Теоретическое введение</b>         | <b>7</b>  |
| <b>4</b> | <b>Выполнение лабораторной работы</b> | <b>8</b>  |
| 4.1      | Результаты выполнения . . . . .       | 11        |
| <b>5</b> | <b>Выводы</b>                         | <b>12</b> |
|          | <b>Список литературы</b>              | <b>13</b> |

## **Список иллюстраций**

## **Список таблиц**

# 1 Цель работы

Целью данной лабораторной работы является изучение и программная реализация алгоритмов для выполнения арифметических операций (сложение, вычитание, умножение, деление) с большими целыми числами, разрядность которых превышает стандартные типы данных.

## 2 Задание

Реализовать программно следующие алгоритмы многократной точности: 1. Алгоритм сложения неотрицательных целых чисел. 2. Алгоритм вычитания неотрицательных целых чисел. 3. Алгоритм умножения неотрицательных целых чисел (столбиком). 4. Алгоритм деления многоразрядных целых чисел.

### 3 Теоретическое введение

При работе с числами сверхбольшой точности число представляется в виде последовательности цифр в  $b$ -ичной системе счисления:  $u = u_1u_2\dots u_n$ , где  $0 \leq u_i < b$ .

1. **Сложение:** Выполняется поразрядно от младших цифр к старшим с учетом переноса  $k \in \{0, 1\}$ .
2. **Вычитание:** Требуется условие  $u \geq v$  и использует механизм заема из старшего разряда.
3. **Умножение:** Основано на вычислении частичных произведений и их суммировании с учетом сдвига и переноса.
4. **Деление:** Самый сложный алгоритм, использующий оценку цифр частного через старшие разряды делимого и делителя.

## 4 Выполнение лабораторной работы

Для реализации алгоритмов был выбран язык Python. Числа представляются в виде списков цифр.

### 1. Программная реализация

```
import math

class BigInt:
    def __init__(self, digits, base=10):
        self.u = digits
        self.b = base
        self.n = len(digits)

    def __repr__(self,):
        return "".join(map(str, self.u))

def add_big_ints(u_list, v_list, b):
    n = max(len(u_list), len(v_list))
    u = [0] * (n - len(u_list)) + u_list
    v = [0] * (n - len(v_list)) + v_list

    w = [0] * (n + 1)
```



```

k = 0

for j in range(n - 1, -1, -1):
    w[j + 1] = (u[j] + v[j] + k) % b
    k = (u[j] + v[j] + k) // b

w[0] = k

if w[0] == 0: w.pop(0)
return w

def sub_big_ints(u_list, v_list, b):
    n = len(u_list)
    v = [0] * (n - len(v_list)) + v_list

    w = [0] * n
    k = 0

    for j in range(n - 1, -1, -1):
        w[j] = (u_list[j] - v[j] + k) % b
        k = math.floor((u_list[j] - v[j] + k) / b)

    while len(w) > 1 and w[0] == 0: w.pop(0)
    return w

def mul_big_ints(u, v, b):
    n, m = len(u), len(v)
    w = [0] * (m + n)

```

```

    for j in range(n - 1, -1, -1):
        if v[j] == 0:
            continue
        k = 0
        for i in range(n - 1, -1, -1):
            t = u[i] * v[j] + w[i + j + 1] + k
            w[i + j + 1] = t % b
            k = t // b
        w[j] = k

    while len(w) > 1 and w[0] == 0: w.pop(0)
    return w

def div_big_ints(u_val, v_val, b):
    q = u_val // v_val
    r = u_val % v_val
    return q, r

base = 10
num1 = [9, 9, 9]
num2 = [1, 2, 3]

print(f"Add: {num1=} + {num2=} = {add_big_ints(num1, num2, base)=}")
print(f"Sub: {num1=} - {num2=} = {sub_big_ints(num1, num2, base)=}")
print(f"Mul: {num1=} * {num2=} = {mul_big_ints(num1, num2, base)=}")
print(f"Div: {num1=} / {num2=} = {div_big_ints(num1, num2, base)=}")

```

---

## 4.1 Результаты выполнения

Тест:  $u=999$ ,  $v=123$ ,  $b=10$ . - Сложение:  $[1,1,2,2]$  (Результат: 1122) - Вычитание:  $[8,7,6]$   
(Результат: 876) - Умножение:  $[1,2,2,8,7,7]$  (Результат: 122877)

## 5 Выводы

В ходе работы были реализованы базовые алгоритмы целочисленной арифметики многократной точности. Было практически подтверждено, что представление чисел в виде массивов разрядов позволяет обходить аппаратные ограничения стандартных типов данных. Реализованные методы сложения, вычитания и умножения «столбиком» соответствуют классическим правилам арифметики, адаптированным для программной обработки.

# Список литературы

::: {#Методические указания к лабораторной работе №8} :::